YachtVersal Production Launch Guide 🚀

📋 Pre-Launch Checklist

Complete this checklist before going live to ensure a smooth launch.

🔐 Phase 1: Security & Environment Setup

1.1 Environment Variables

Create production `.env` file:

```bash
```

```
# NEVER commit this file to Git!
# Store securely in your hosting provider's environment variables

# Database
DATABASE_URL=postgresql://user:password@host:5432/yachtversal_prod
POSTGRES_USER=yachtversal_prod_user
POSTGRES_PASSWORD=STRONG_RANDOM_PASSWORD_HERE
POSTGRES_DB=yachtversal_prod

# Backend API
SECRET_KEY=GENERATE_STRONG_SECRET_KEY_HERE  # Use: openssl rand -hex 32
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=43200  # 30 days

# Frontend
NEXT_PUBLIC_API_URL=https://api.yachtversal.com
NEXT_PUBLIC_SITE_URL=https://www.yachtversal.com

# Email Service (choose one)
# SendGrid
SENDGRID_API_KEY=SG.xxx
SENDGRID_FROM_EMAIL=noreply@yachtversal.com

# OR AWS SES
AWS_ACCESS_KEY_ID=xxx
AWS_SECRET_ACCESS_KEY=xxx
AWS_REGION=us-east-1
SES_FROM_EMAIL=noreply@yachtversal.com

# Stripe (Production Keys!)
STRIPE_SECRET_KEY=sk_live_xxx  # NOT sk_test!
STRIPE_PUBLISHABLE_KEY=pk_live_xxx  # NOT pk_test!
STRIPE_WEBHOOK_SECRET=whsec_xxx
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=pk_live_xxx

# Cloudflare R2 Storage
R2_ENDPOINT_URL=https://YOUR_ACCOUNT_ID.r2.cloudflarestorage.com
R2_ACCESS_KEY_ID=xxx
R2_SECRET_ACCESS_KEY=xxx
R2_BUCKET_NAME=yachtversal-media-prod
R2_PUBLIC_URL=https://media.yachtversal.com

# Optional: Analytics
```

```
GOOGLE_ANALYTICS_ID=G-XXXXXXXXXX
MIXPANEL_TOKEN=xxx

# Optional: Error Tracking
SENTRY_DSN=https://xxx@sentry.io/xxx

# Rate Limiting
REDIS_URL=redis://localhost:6379/0  # Or managed Redis

# CORS (adjust as needed)
ALLOWED_ORIGINS=https://www.yachtversal.com,https://yachtversal.com
```

1.2 Generate Secure Secrets

```bash
# Generate SECRET_KEY
python -c "import secrets; print(secrets.token_hex(32))"

# Generate strong database password
python -c "import secrets; print(secrets.token_urlsafe(32))"
```

1.3 Security Headers

Update CORS settings in `app/main.py`:

```python
app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        "https://www.yachtversal.com",
        "https://yachtversal.com"
    ],  # REMOVE "*" wildcard!
    allow_credentials=True,
    allow_methods=["GET", "POST", "PUT", "DELETE", "PATCH"],
    allow_headers=["*"],
)
```

Add security headers:

```python
```

```
from fastapi.middleware.trustedhost import TrustedHostMiddleware
from fastapi.middleware.httpsredirect import HTTPSRedirectMiddleware

# Force HTTPS
app.add_middleware(HTTPSRedirectMiddleware)

# Restrict hosts
app.add_middleware(
    TrustedHostMiddleware,
    allowed_hosts=["yachtversal.com", "*.yachtversal.com"]
)
```

1.4 Update Frontend Security

Update `next.config.js`:

```javascript
```

```
module.exports = {
  // Security headers
  async headers() {
    return [
      {
        source: '/(.*)',
        headers: [
          {
            key: 'X-Frame-Options',
            value: 'DENY'
          },
          {
            key: 'X-Content-Type-Options',
            value: 'nosniff'
          },
          {
            key: 'Referrer-Policy',
            value: 'strict-origin-when-cross-origin'
          },
          {
            key: 'Permissions-Policy',
            value: 'camera=(), microphone=(), geolocation=()'
          }
        ]
      }
    ]
  },

  // Enable image optimization
  images: {
    domains: ['media.yachtversal.com', 'YOUR_ACCOUNT_ID.r2.cloudflarestorage.com'],
    formats: ['image/avif', 'image/webp']
  }
}
```

💾 Phase 2: Database Setup

2.1 Create Production Database

Option A: Managed PostgreSQL (Recommended)

- **Render**: Database tab → New PostgreSQL
- **AWS RDS**: Create PostgreSQL instance
- **DigitalOcean**: Managed Databases
- **Supabase**: Free tier with PostgreSQL

Option B: Self-Hosted

```bash
# On your server
sudo apt update
sudo apt install postgresql postgresql-contrib
sudo systemctl start postgresql
sudo systemctl enable postgresql

# Create database
sudo -u postgres psql
CREATE DATABASE yachtversal_prod;
CREATE USER yachtversal_user WITH PASSWORD 'your_secure_password';
GRANT ALL PRIVILEGES ON DATABASE yachtversal_prod TO yachtversal_user;
\q
```

2.2 Run Database Migrations

```bash
# Connect to production database
export DATABASE_URL="postgresql://user:pass@host:5432/yachtversal_prod"

# Run migrations
alembic upgrade head

# Verify tables created
psql $DATABASE_URL -c "\dt"
```

2.3 Create Admin User

```python
```

```
# Create admin via Python script
from app.db.session import SessionLocal
from app.models.user import User
from passlib.context import CryptContext

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

db = SessionLocal()

admin = User(
    email="admin@yachtversal.com",
    password_hash=pwd_context.hash("YOUR_SECURE_PASSWORD"),
    first_name="Admin",
    last_name="User",
    user_type="admin",
    active=True,
    verified=True,
    email_verified=True
)

db.add(admin)
db.commit()
print(f"Admin created: {admin.email}")
```

2.4 Database Backups

Set up automated backups:

```bash
# Cron job for daily backups (add to crontab)
0 2 * * * pg_dump $DATABASE_URL | gzip > /backups/yachtversal_$(date +\%Y\%m\%d).sql.gz

# Keep only last 30 days
0 3 * * * find /backups -name "yachtversal_*.sql.gz" -mtime +30 -delete
```

☁️ Phase 3: Cloud Infrastructure Setup

3.1 Cloudflare R2 Setup

1. **Create R2 Bucket:**

- Go to Cloudflare Dashboard → R2
- Create bucket: `yachtversal-media-prod`
- Set to Public

2. **Generate API Tokens:**
   - Manage R2 API Tokens → Create Token
   - Copy credentials to `.env`
3. **Custom Domain:**
   - Add custom domain: `media.yachtversal.com`
   - Update DNS: CNAME to R2 bucket
   - Update `R2_PUBLIC_URL` in `.env`
4. **CORS Configuration:**

```json
[
  {
    "AllowedOrigins": ["https://yachtversal.com", "https://www.yachtversal.com"],
    "AllowedMethods": ["GET", "PUT", "POST", "DELETE"],
    "AllowedHeaders": ["*"],
    "MaxAgeSeconds": 3600
  }
]
```

3.2 Stripe Production Setup

1. **Activate Live Mode:**
   - Stripe Dashboard → Activate Account
   - Complete business verification
   - Add bank account for payouts
2. **Get Production Keys:**
   - Developers → API Keys
   - Copy `sk_live_...` and `pk_live_...`
   - Update `.env` files
3. **Configure Webhooks:**

- Developers → Webhooks → Add endpoint
- URL: `https://api.yachtversal.com/stripe/webhook`
- Events to listen for:
  - `payment_intent.succeeded`
  - `payment_intent.payment_failed`
  - `customer.subscription.created`
  - `customer.subscription.updated`
  - `customer.subscription.deleted`
- Copy webhook secret to `.env`

4. **Test Webhooks:**

```bash
stripe listen --forward-to https://api.yachtversal.com/stripe/webhook
```

3.3 Email Service Setup

Option A: SendGrid (Recommended for small-medium scale)

1. Create account at sendgrid.com
2. Verify sender email/domain
3. Generate API key
4. Add to `.env`

Option B: AWS SES (Better for high volume)

1. Create AWS account
2. SES → Verify domain
3. Request production access (remove sandbox)
4. Create IAM user with SES permissions
5. Add credentials to `.env`

3.4 Domain & DNS Setup

Buy domain (if not done):

- Namecheap, GoDaddy, or Cloudflare Registrar

DNS Configuration:

```
# A Records
yachtversal.com          → YOUR_SERVER_IP
www.yachtversal.com      → YOUR_SERVER_IP
api.yachtversal.com      → YOUR_API_SERVER_IP

# CNAME Records
media.yachtversal.com    → R2_BUCKET_URL
```

SSL Certificates:

- Use Cloudflare (automatic)
- Or Let's Encrypt (free)
- Or managed by hosting provider

🚀 Phase 4: Deployment

4.1 Backend Deployment

Option A: Render (Easiest)

1. Connect GitHub repo
2. New Web Service
3. Build command: `pip install -r requirements.txt`
4. Start command: `uvicorn app.main:app --host 0.0.0.0 --port $PORT`
5. Add environment variables
6. Deploy!

Option B: DigitalOcean App Platform

1. Create new app from GitHub
2. Configure as Python app
3. Add environment variables

4. Deploy

Option C: AWS EC2 (Most Control)

```bash
# On EC2 instance
sudo apt update
sudo apt install python3-pip python3-venv nginx

# Clone repo
git clone https://github.com/yourusername/yachtversal-backend.git
cd yachtversal-backend

# Create virtual environment
python3 -m venv venv
source venv/bin/activate

# Install dependencies
pip install -r requirements.txt

# Install Gunicorn
pip install gunicorn

# Create systemd service
sudo nano /etc/systemd/system/yachtversal.service
```

systemd service file:

```ini
```

```
[Unit]
Description=YachtVersal API
After=network.target

[Service]
User=ubuntu
WorkingDirectory=/home/ubuntu/yachtversal-backend
Environment="PATH=/home/ubuntu/yachtversal-backend/venv/bin"
EnvironmentFile=/home/ubuntu/yachtversal-backend/.env
ExecStart=/home/ubuntu/yachtversal-backend/venv/bin/gunicorn app.main:app -w 4 -k uvicorn.workers.UvicornWorker -b 0.

[Install]
WantedBy=multi-user.target
```

```bash
# Start service
sudo systemctl start yachtversal
sudo systemctl enable yachtversal
sudo systemctl status yachtversal

# Configure Nginx
sudo nano /etc/nginx/sites-available/yachtversal
```

Nginx configuration:

```nginx
server {
    listen 80;
    server_name api.yachtversal.com;

    location / {
        proxy_pass http://localhost:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

```bash
```

```
# Enable site and restart Nginx
sudo ln -s /etc/nginx/sites-available/yachtversal /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl restart nginx

# Get SSL certificate
sudo apt install certbot python3-certbot-nginx
sudo certbot --nginx -d api.yachtversal.com
```

4.2 Frontend Deployment

Option A: Vercel (Recommended for Next.js)

1. Import GitHub repo to Vercel
2. Framework: Next.js
3. Add environment variables
4. Deploy!
5. Add custom domain

Option B: Netlify

1. Connect GitHub repo
2. Build command: `npm run build`
3. Publish directory: `.next`
4. Add environment variables
5. Deploy

Option C: Self-Hosted

```bash
```

```
# On server
git clone https://github.com/yourusername/yachtversal-frontend.git
cd yachtversal-frontend

# Install dependencies
npm install

# Build for production
npm run build

# Install PM2 for process management
npm install -g pm2

# Start with PM2
pm2 start npm --name "yachtversal" -- start
pm2 save
pm2 startup
```

🔄 Phase 5: Post-Deployment Tasks

5.1 Set Up Monitoring

Backend Monitoring:

```bash
# Install Sentry (error tracking)
pip install sentry-sdk

# In app/main.py
import sentry_sdk

sentry_sdk.init(
    dsn=os.getenv("SENTRY_DSN"),
    environment="production",
    traces_sample_rate=0.1
)
```

Uptime Monitoring:

- UptimeRobot (free)

- Pingdom
- Better Uptime

Add checks for:

- https://api.yachtversal.com/health
- https://www.yachtversal.com

5.2 Configure Analytics

Google Analytics 4:

```javascript
// In app/layout.tsx
import Script from 'next/script'

<Script
  src={`https://www.googletagmanager.com/gtag/js?id=${process.env.NEXT_PUBLIC_GA_ID}`}
  strategy="afterInteractive"
/>
<Script id="google-analytics" strategy="afterInteractive">
  {`
    window.dataLayer = window.dataLayer || [];
    function gtag(){dataLayer.push(arguments);}
    gtag('js', new Date());
    gtag('config', '${process.env.NEXT_PUBLIC_GA_ID}');
  `}
</Script>
```

5.3 Set Up Cron Jobs

```bash
```

```
# Add to crontab (crontab -e)

# Expire featured listings every hour
0 * * * * curl -X POST https://api.yachtversal.com/api/admin/featured-listings/expire-check -H "Authorization: Bearer ADMI

# Send daily notification emails at 9 AM
0 9 * * * /usr/bin/python3 /path/to/send_daily_emails.py

# Clean up old sessions weekly
0 0 * * 0 /usr/bin/python3 /path/to/cleanup_sessions.py

# Database backup daily at 2 AM
0 2 * * * pg_dump $DATABASE_URL | gzip > /backups/db_$(date +\%Y\%m\%d).sql.gz
```

5.4 Create Monitoring Dashboard

Key metrics to track:

- Active users (daily/weekly/monthly)
- New listings created
- Featured listing purchases
- API response times
- Error rates
- Database query performance
- Storage usage

Use:

- Grafana + Prometheus (self-hosted)
- Datadog (managed)
- New Relic (managed)

📝 Phase 6: Legal & Compliance

6.1 Legal Pages

Create these pages **before launch**:

- **Privacy Policy** (`/privacy`)
  - Data collection practices
  - Cookie usage
  - Third-party services (Stripe, analytics)
  - User rights (GDPR, CCPA)
- **Terms of Service** (`/terms`)
  - User responsibilities
  - Listing guidelines
  - Payment terms
  - Liability limitations
- **Cookie Policy** (`/cookies`)
  - Types of cookies used
  - How to disable cookies

Resources:

- Use Termly.io or TermsFeed for templates
- Have lawyer review if budget allows

6.2 GDPR Compliance

If serving EU users:

```typescript
// Cookie consent banner
import CookieConsent from 'react-cookie-consent';

<CookieConsent
  location="bottom"
  buttonText="Accept"
  cookieName="yachtversal-consent"
  expires={365}
>
  We use cookies to enhance your experience.
  <a href="/privacy">Learn more</a>
</CookieConsent>
```

Implement:

- Data export functionality
- Data deletion on request
- Cookie consent management
- Privacy policy compliance

6.3 Payment Compliance

Stripe handles:

- PCI compliance
- Fraud detection
- 3D Secure

Your responsibility:

- Never store card numbers
- Use Stripe's hosted forms
- Implement proper error handling
- Refund policy

🧪 Phase 7: Pre-Launch Testing

7.1 Functionality Testing

Test every major flow:

- [ ] User registration (all types: dealer, user, salesman)
- [ ] Email verification
- [ ] Password reset
- [ ] Login/logout
- [ ] Create listing
- [ ] Edit listing
- [ ] Delete listing
- [ ] Upload images
- [ ] Feature listing purchase (with test card)

- [ ] Search functionality
- [ ] Filtering & sorting
- [ ] Saved listings
- [ ] Price alerts
- [ ] Messaging system
- [ ] Admin dashboard functions
- [ ] Dealer dashboard functions
- [ ] Blog post creation
- [ ] Mobile responsiveness

7.2 Performance Testing

```bash
# Load testing with Apache Bench
ab -n 1000 -c 10 https://api.yachtversal.com/api/listings

# Or use k6
k6 run load-test.js
```

Target metrics:

- Page load: < 3 seconds
- API response: < 200ms
- Time to interactive: < 5 seconds
- Lighthouse score: > 90

7.3 Security Testing

Run security scans:

```bash
# OWASP ZAP for vulnerability scanning
docker run -t owasp/zap2docker-stable zap-baseline.py -t https://www.yachtversal.com

# Check SSL configuration
https://www.ssllabs.com/ssltest/analyze.html?d=yachtversal.com

# Check security headers
https://securityheaders.com/?q=yachtversal.com
```

7.4 Browser Testing

Test on:

- [ ] Chrome (desktop & mobile)
- [ ] Safari (desktop & mobile)
- [ ] Firefox
- [ ] Edge
- [ ] Mobile browsers (iOS Safari, Chrome)

📢 Phase 8: Launch Strategy

8.1 Soft Launch

Week 1: Limited Beta

- Invite 10-20 friendly dealers
- Monitor for bugs
- Gather feedback
- Fix critical issues

Week 2: Expanded Beta

- Open to 50-100 dealers
- Implement feedback
- Test at scale
- Monitor performance

8.2 Marketing Preparation

Before public launch:

- [ ] Create launch announcement
- [ ] Prepare social media posts
- [ ] Set up email list
- [ ] Create demo videos
- [ ] Write press release
- [ ] Contact yacht industry publications

8.3 Launch Day Checklist

Morning of launch:

- [ ] Final database backup
- [ ] Verify all systems operational
- [ ] Check monitoring dashboards
- [ ] Test critical user flows one more time
- [ ] Have rollback plan ready

During launch:

- [ ] Post on social media
- [ ] Send announcement emails
- [ ] Monitor error logs
- [ ] Watch performance metrics
- [ ] Be ready for hot fixes

End of day:

- [ ] Review analytics
- [ ] Check error rates
- [ ] Gather initial feedback
- [ ] Celebrate! 🎉

🛠️ Phase 9: Post-Launch

9.1 Week 1 Priorities

- Monitor error rates closely
- Respond to user feedback quickly
- Fix critical bugs immediately
- Scale resources if needed
- Send thank you emails to early users

9.2 Ongoing Maintenance

Daily:

- Check error logs

- Monitor uptime
- Review user feedback

Weekly:

- Analyze user metrics
- Review performance
- Update content (blog posts)
- Plan feature improvements

Monthly:

- Security updates
- Database optimization
- Review hosting costs
- Analyze conversion rates
- Plan next features

9.3 Scaling Plan

When to scale:

- API response times > 500ms
- Database CPU > 80%
- Storage > 80% capacity
- Too many concurrent users

Scaling options:

- Upgrade server resources
- Add database replicas
- Implement Redis caching
- Use CDN for static assets
- Consider serverless functions

📊 Success Metrics

Track these KPIs post-launch:

Week 1:

- Signups: 50+ dealers
- Listings: 100+
- Zero critical bugs

Month 1:

- MAU: 500+
- Listings: 1,000+
- Featured purchases: 10+
- <1% error rate

Month 3:

- MAU: 2,000+
- Listings: 5,000+
- Revenue: $5,000+/month
- SEO: Page 1 for "yacht marketplace"

🆘 Emergency Procedures

Rollback Plan

```bash
# Backend rollback
git checkout previous_stable_commit
git push -f origin main
# Trigger redeploy

# Database rollback
psql $DATABASE_URL < /backups/latest_backup.sql
```

Emergency Contacts

Create list of:

- Hosting provider support
- Database provider support
- Stripe support
- Your team members
- Technical advisor/consultant

Common Issues & Fixes

Site down:

1. Check server status
2. Check DNS
3. Check SSL certificate
4. Review error logs
5. Rollback if needed

Database issues:

1. Check connection pool
2. Review slow queries
3. Restart database
4. Restore from backup if corrupted

Payment issues:

1. Check Stripe dashboard
2. Verify webhook is working
3. Review Stripe logs
4. Contact Stripe support

✅ Final Launch Checklist

- [ ] All environment variables set
- [ ] Database migrated and backed up
- [ ] Admin user created
- [ ] SSL certificates active
- [ ] Domain DNS configured
- [ ] Cloudflare R2 configured
- [ ] Stripe production mode active
- [ ] Email service configured
- [ ] Monitoring tools active
- [ ] Analytics installed
- [ ] Legal pages published
- [ ] Cookie consent implemented
- [ ] All tests passing
- [ ] Performance optimized
- [ ] Security scan clean
- [ ] Backup systems running
- [ ] Cron jobs scheduled
- [ ] Error tracking active
- [ ] Team trained
- [ ] Support channels ready
- [ ] Marketing materials prepared

🎉 You're Ready to Launch!

Remember:

- **Perfection isn't required -** ship and iterate
- **Monitor closely** the first 48 hours
- **Be responsive** to early user feedback
- **Celebrate wins -** launching is a huge milestone!

Good luck! 🚀

Need help? Set up monitoring alerts to notify you immediately of:

- Server downtime

- Error spikes
- Payment failures
- Database issues

You've got this! 💪